NATIONAL DIGITAL LIBRARY OF INDIA (NDLI)

Central Administration Office • Indian Institute of Technology Kharagpur

ENTERPRISE TECHNICAL REFERENCE MANUAL

Architecture & Handover Specification v5.0

NDLI Club Management System: Technical Architecture, Security & Engineering Manual

An Exhaustive Technical Specification Document for Enterprise Governance, Cryptographic Security, Zero-Dependency Flat-File CSV Concurrency, Master-Worker Node Synchronization, REST API Catalog, and Cloud Deployment Architecture for Data Transparency & Client Handover.

Python 3.10+ Native Kernel

PBKDF2-HMAC-SHA256 (100k rounds)

Atomic CSV Mutex Engine

Dual-Layer Security Restore

7 Regional Worker Nodes

Google Apps Script Webhook Relay

35+ REST API Endpoints

Modern Minimalist UI v3.0

DOCUMENT RELEASE & STATUS

Confidential Enterprise Technical Guide • September 2026

COMPLIANCE & HANDOVER SCOPE

Client Handover, Data Transparency, Code Audit & Maintenance

LEAD SYSTEM ARCHITECT & FULL-STACK DEVELOPER

Dr. Anirban Mukherjee

Midnapore, WB 721101, India

Email: anirbanmid@gmail.com

Call: +91 7797387474

Comprehensive Technical Navigation Index

An exhaustive architectural reference index across all 20 technical sections, security models, and code implementations:

★ System Overview & End-to-End Process Flow (Start Here) Page 3
★ System Mindmap & Manual Map Page 4
1. Executive Technical Summary & Architectural Philosophy Page 5
2. System Topology & Multi-Tier Architecture Page 6
3. Cryptographic Security & Authentication Subsystem Page 7
4. Dual-Layer Security Challenge for Disaster Recovery Page 8
5. Database Schemas & Flat-File Storage Engine Page 9
6. Flat-File ACID Simulation & Mutex Concurrency Engine Page 10
7. Core Backend Codebase & Module Specifications Page 11
8. Synchronization & Node-to-Master Reconciliation Page 12
9. Mathematical Models & Analytical Formulas Page 13
10. Geopolitical State-Zone Mapping Specification Page 14
11. REST API Specification (Part 1: Auth & Operations) Page 15
12. REST API Specification (Part 2: Admin & System) Page 16
13. Backup Architecture & Disaster Recovery Pipeline Page 17
14. Emergency Restore Protocol & CLI Fallback Page 18
15. Front-End Architecture & Design System v3.0 Page 19
16. Cloud Deployment & DevOps Infrastructure Page 20
17. Automated Quality Assurance & Test Architecture Page 21
18. System Maintenance, Operations & Troubleshooting Runbook Page 22
19. Security Audit, Data Integrity & Compliance Page 23
20. System Handover Sign-Off & Architectural Attribution Page 24
Appendix A. Auto-Generated API Reference & Architecture Map Page 27

System Overview & End-to-End Process Flow (Start Here)

This page answers the single most important question in the manual: **how does the entire system execute one unit of work, end to end?** Every business action — an activity logged, a club approved, a certificate issued — travels the same pipeline shown below: client tier → Python engine tier → atomic flat-file storage tier mirrored to Google Drive.

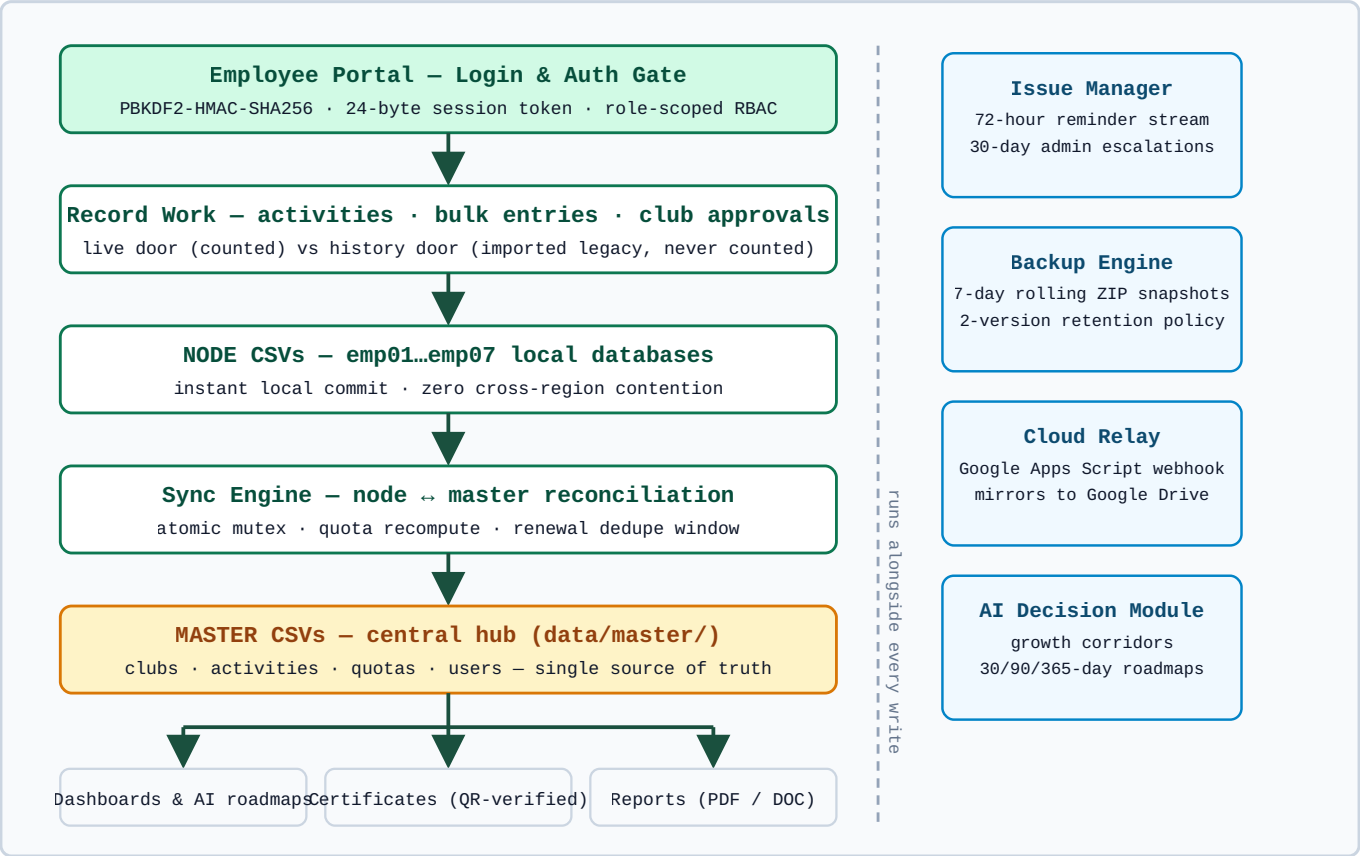


Figure 0.1 — End-to-end process flow: one unit of work travels login → record → node CSVs → sync → master CSVs → outputs, while four continuous services (issues, backups, cloud relay, AI) run alongside every write.

System Mindmap & Manual Map

The mindmap groups every subsystem into its tier; the manual map table after it jumps straight to the chapter that answers your question.

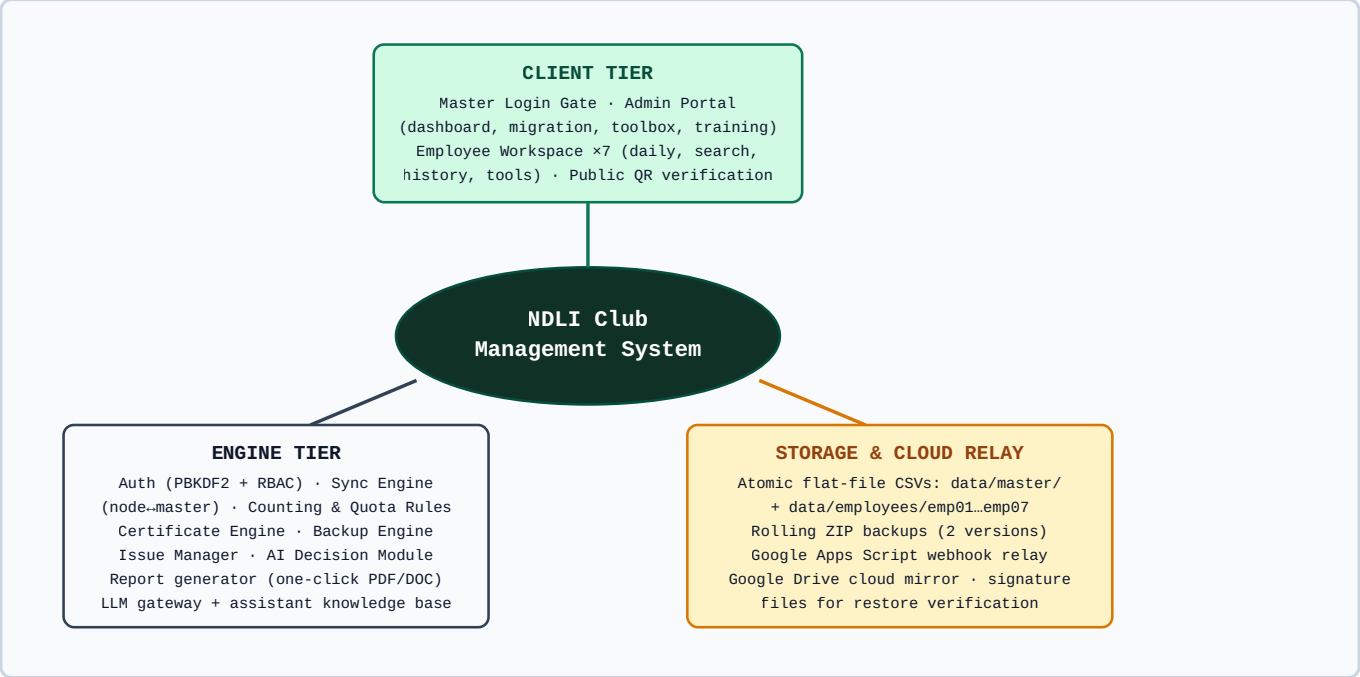


Figure 0.2 — System mindmap: the three tiers and every subsystem inside them.

Manual Map — which chapter answers your question

If you want to know…	Read
How one unit of work executes end to end	This page (Figure 0.1)
Architecture, tiers, and which engines exist	Ch. 2 + Figure 0.2
Password hashing, sessions, RBAC	Ch. 3
Disaster-recovery security challenge	Ch. 4
Database schemas and the flat-file engine	Ch. 5–6
Backend modules and code specifications	Ch. 7
Node-to-master synchronization	Ch. 8
Quota formulas and analytics math	Ch. 9
State-zone mapping (36 states → 6 zones)	Ch. 10
REST API contracts (auth, admin, system)	Ch. 11–12 + Appendix A
Backups and the disaster-recovery pipeline	Ch. 13–14
Front-end architecture and design system	Ch. 15
Cloud deployment and DevOps	Ch. 16
Testing and the QA architecture	Ch. 17
Operations runbook and troubleshooting	Ch. 18
Security audit and compliance	Ch. 19
Handover sign-off and attribution	Ch. 20
Day-to-day operating how-tos (step by step)	The User Manual

1. Executive Technical Summary & Architectural Philosophy

1.1 Project Operational Scope & Mandate

The **National Digital Library of India (NDLI) Club Management and Employee Activity Tracking System** is an enterprise web platform commissioned to manage, track, and audit institutional clubs nationwide across all 36 States and Union Territories. Coordinated under the Ministry of Education, Government of India, and governed by the Central Administration Office at the **Indian Institute of Technology Kharagpur (IIT Kharagpur)**, the system bridges 7 Regional Zonal Officers with central executive leadership.

1.2 Core Architectural Principles & Design Decisions

⚙️ Zero External Dependency Kernel

Engineered entirely using Python standard library primitives (`http.server`, `csv`, `hashlib`, `hmac`, `secrets`, `threading`, `json`). Requires no third-party package runtime (e.g., Django, Flask, or SQLAlchemy), eliminating dependency hell and container CVE surface.

🔒 Multi-Tiered Cryptographic Protection

Passwords encrypted using PBKDF2-HMAC-SHA256 with 100,000 rounds and 16-byte random salts. Sensitive actions (such as Emergency Restore) require a Dual-Layer Security Challenge (dynamic Canvas CAPTCHA + Master Admin password re-authentication).

🗄️ Flat-File Atomic CSV Database Engine

Bypasses complex database servers (PostgreSQL/MySQL) by implementing thread-safe atomic file mutex locks with temporary file write-and-replace semantics (`os.replace`) guaranteeing ACID-like consistency on plain-text CSV stores.

🌐 Hybrid Cloud / Serverless Webhook Mirroring

Combines a primary Python container service (Render/Docker) with an automated Google Drive / Google Apps Script (GAS) Webhook Relay pipeline, executing real-time cloud data mirroring and 7-day rolling ZIP archives.

1.3 Purpose of this Technical Manual

This manual is compiled for **technical handover, source code audits, and operational transparency**. It provides full disclosure of underlying algorithms, cryptographic implementations, directory hierarchies, schema definitions, REST API contracts, and disaster recovery procedures.

2. System Topology & Multi-Tier Architecture

2.1 High-Level Component Topology

The architecture is split into three discrete tiers: the **Client Presentation Tier** (HTML5/Vanilla JS), the **Application Engine Tier** (Python Native HTTP Server), and the **Storage & Backup Tier** (Local Atomic CSVs + Google Drive Cloud Relay):

```
+-----------------------------------------------------------------------------------------------+ | CLIENT PRESENTATION TIER | | [Master Login Gate] | [Admin Office Portal] | [7 Regional Employee Workspaces (EMP01-EMP07)] | | - Vanilla JS ES6+ | - Real-time SVG Charts | - 4 Dynamic Tabs (Daily, Search, History, Tools) | | - Glass Morphism UI | - AI Decision Module | - Integrated 12-Tool Box (osTicket first) | +-----------------------------------------------+-----------------------------------------------+ | HTTP / HTTPS REST APIs (JSON) v +-----------------------------------------------------------------------------------------------+ | APPLICATION ENGINE TIER | | app.py (HTTP Server) | | +-------------------------------------------------------------------------------------------+ | | | Security & Auth Engine (auth.py): PBKDF2-HMAC-SHA256, 16-byte Salt, 24-byte Session Tokens | | | | State-to-Zone Engine (state_zone_mapper.py): 36 States/UTs to 6 Geopolitical Zones | | | | Sync Engine (sync_engine.py): Node-to-Master Two-Way Reconciliation & Quota Recalculation | | | | Backup Engine (backup_engine.py): 7-Day Rolling ZIP Snapshots & 2-Version Retention Policy | | | | Issue Manager (issue_manager.py): 72-Hour Reminder Stream & 30-Day Admin Escalations | | | | AI Heuristic Engine (decision_module.py): Growth Corridors & 30/90/365-Day Roadmaps | | | +-------------------------------------------------------------------------------------------+ | +-----------------------------------------------+-----------------------------------------------+ | Atomic File Mutex (threading.Lock) v +-----------------------------------------------------------------------------------------------+ | STORAGE & CLOUD RELAY TIER | | LOCAL ATOMIC CSV STORAGE: GOOGLE DRIVE CLOUD RELAY: | | data/master/ (Clubs, Activities, Quotas, Users) --> Google Apps Script Webhook Relay (Code.gs) | | data/employees/emp01..07/ (Node DBs) --> Automated Mirroring into Google Drive /backups | | data/backups/ (Rolling ZIP Archives, Max 2) --> 24x7 Real-Time Cloud-to-Drive Sync | +-----------------------------------------------------------------------------------------------+
```

2.2 Node vs. Master Database Separation

- **Worker Nodes (`data/employees/emp01/` to `emp07/`):** Each zonal officer operates independently. Field activities and club approvals are committed immediately to local node CSVs, ensuring zero cross-region contention.
- **Master Central Hub (`data/master/`):** Aggregates all node records in real-time. Governed by atomic mutex locks to guarantee data consistency during concurrent writes.

3. Cryptographic Security & Authentication Subsystem

3.1 Password Encryption: PBKDF2-HMAC-SHA256

User credentials stored in `data/master/master_users.csv` and regional `credentials.csv` are never stored in cleartext. The system employs **Password-Based Key Derivation Function 2 (PBKDF2)** using **HMAC-SHA256**:

```
DerivedKey = PBKDF2(PRF = HMAC-SHA256, Password, Salt = 16_bytes_random, Iterations = 100,000, dkLen = 32_bytes)
```

```
# Implementation excerpt from auth.py
def hash_password(password: str, salt: bytes = None) -> tuple[str, str]:
    if salt is None:
        salt = secrets.token_bytes(16)
    key = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode('utf-8'),
        salt,
        100000,
        dklen=32
    )
    return key.hex(), salt.hex()

def verify_password(stored_hash_hex: str, stored_salt_hex: str, password_attempt: str) -> bool:
    salt = bytes.fromhex(stored_salt_hex)
    attempt_key = hashlib.pbkdf2_hmac('sha256', password_attempt.encode('utf-8'), salt, 100000, dklen=32)
    return hmac.compare_digest(stored_hash_hex, attempt_key.hex()) # Constant-time comparison
```

3.2 Session Entropy & Token Management

- **Token Generation:** Tokens are 24 cryptographically random bytes generated via `secrets.token_hex(24)` (yielding 48 hexadecimal characters, 192 bits of entropy).
- **Timing Attack Resistance:** Hash comparisons use `hmac.compare_digest()` to ensure constant-time execution, mitigating side-channel timing analysis.
- **Session Segregation:** Client sessions are compartmentalized into distinct keys in `sessionStorage`: `ndli_token` (generic auth), `ndli_admin_token` (master control), and `ndli_employee_session` (zonal node context).

3.3 Role-Based Access Control (RBAC) Specification

Role / Persona	Identifier	Permitted Scope	Restricted Endpoints
Master Admin	`admin@iitkgp.ac.in`	Full system control, employee provisioning, master CSV download, database rollback, AI policy roadmaps.	None. Full root authority.
Regional Zonal Officers	`EMP01` to `EMP07`	Daily SEC A logging, SEC B/C club approvals, node CSV downloads, local issue reporting, Tool Box utilities.	Cannot provision users, cannot trigger database restore, cannot modify other zones.

4. Dual-Layer Security Challenge for Disaster Recovery

4.1 Threat Model & Operational Rationale

An emergency database restore is an inherently destructive, non-reversible operation that overwrites both Central Master CSVs and all 7 Regional Node CSVs with historical snapshot data. To protect against accidental clicks, cross-site request forgery (CSRF), and unauthorized workstation access during active admin sessions, the system implements a **Dual-Layer Security Challenge**:

```
RESTORE REQUEST TRIGGERED (Admin clicks "One-Click Restore Database") | v +-----------------------------------------------------------------------+ | LAYER 1: Dynamic Anti-Bot Algorithmic Challenge | | - 5-Character randomized alphanumeric token (A-Z, 2-9, excluding 0/O/1/I) | | - Distorted vector lines & background noise on HTML5 Canvas | | - Token validated server-side; expires upon modal closure | +-----------------------------------------------------------------------+ | (Layer 1 Passed) v +-----------------------------------------------------------------------+ | LAYER 2: Master Administrator Re-Authentication | | - Requires re-entering Master Admin Password (admin@iitkgp.ac.in) | | - PBKDF2-HMAC-SHA256 salt verification against master_users.csv | | - Prevents unauthorized execution via hijacked browser sessions | +-----------------------------------------------------------------------+ | (Layer 2 Passed) v +-----------------------------------------------------------------------+ | RESTORATION EXECUTION ENGINE | | 1. Unpack ZIP archive to temporary directory | | 2. Perform atomic file replacement (os.replace) on master & nodes | | 3. Execute automatic Node-to-Master Reconciliation | | 4. Flush in-memory metrics cache & reload dashboard telemetry | +-----------------------------------------------------------------------+
```

4.2 Server-Side Verification Endpoint Contract (`POST /api/admin/backup/restore`)

- **Payload Schema:** `{"password": "<master_pwd>", "backup_id": "<optional_snapshot_id>"}`
- **Validation Logic:** If password hash does not match Master Admin salt record, an HTTP `403 Forbidden` is returned.
- **Post-Restore Reconciliation:** Immediately invokes `sync_engine.reconcile_all_nodes()` to restore synchronized quota tallies across all regions.

5. Database Schemas & Flat-File Storage Engine

5.1 Master Database Schema Dictionaries

All database stores are maintained in comma-separated value (CSV) tables adhering to strict RFC 4180 formatting:

1. `master_clubs.csv` (Central Club Registry)

Column	Data Type	Constraint & Description
`id`	`STRING`	Primary Key. Format: `CLUB-[STATE]-[YEAR]-[NUM]`. Unique index.
`reg_no`	`STRING`	Official institutional registration identifier.
`name`	`STRING`	Full verified institution name.
`state`	`STRING`	State or Union Territory (must match 36 statutory territories).
`zone`	`STRING`	Geopolitical zone (North, Central, West, East, North East, South).
`patron_email`	`STRING`	Email of Head of Institution (Principal/Vice-Chancellor).
`president_email`	`STRING`	Email of Faculty Lead / NDLI Club Coordinator.
`secretary_email`	`STRING`	Email of Student Representative / Member Secretary.
`approved_by`	`STRING`	Foreign Key → `master_users.id` (e.g., `EMP01`).
`approval_date`	`ISO-DATE`	Date of Approval (YYYY-MM-DD).
`last_renewal_date`	`ISO-DATE`	Date of latest registration renewal.
`renewal_date`	`ISO-DATE`	Next renewal deadline. Invariant: `DOA + 1 year` or `LRD + 1 year`.

2. `master_activities.csv` (Field Interaction Ledger)

Column	Data Type	Constraint & Description
`id`	`STRING`	Primary Key. Unique activity identifier (UUID or timestamp-hash).
`emp_id`	`STRING`	Foreign Key → `master_users.id` (originating officer).
`timestamp`	`ISO-TIMESTAMP`	Execution timestamp with IST timezone notation.
`support_type`	`ENUM`	`Phone call`, `Closing of OS Ticket`, `Online training`, `Offline training`.
`notes`	`STRING`	Institutional stakeholder notes and agenda details.
`state / zone`	`STRING`	Geopolitical territory where support was rendered.

3. `master_quotas.csv` (Officer Performance Quotas)

Column	Data Type	Constraint & Description
`emp_id`	`STRING`	Primary Key → `EMP01` through `EMP07`.
`clubs_approved`	`INTEGER`	Cumulative verified clubs authorized by officer.
`total_activities`	`INTEGER`	Total support interactions submitted to date.
`last_activity`	`ISO-TIMESTAMP`	Timestamp of most recent entry.

6. Flat-File ACID Simulation & Mutex Concurrency Engine

6.1 Concurrency Architecture: Thread-Safe Mutex Locking

In high-concurrency environments with multiple regional officers logging data simultaneously, standard file writes risk race conditions and torn writes. The system implements a dedicated **Atomic CSV Engine (`db/csv_engine.py`)** guaranteeing ACID properties on flat files:

```
CONCURRENT WRITE REQUEST (Worker Thread) | v [Acquire threading.Lock() for specific file path] | v [Read existing CSV content into in-memory dictionary cache] | v [Apply row insertions / upsert modifications] | v [Write new content to temporary scratch file: filename.csv.tmp.PID] | v [Flush to disk buffer via os.fsync(fd)] | v [Execute atomic filesystem rename: os.replace(tmp_file, target_file)] | v [Release threading.Lock()] ---> WRITE TRANSACTION COMMITTED SAFELY
```

6.2 ACID Property Implementation Matrix

ACID Property	Standard RDBMS Mechanism	NDLI Flat-File CSV Engine Implementation
Atomicity	Write-Ahead Log (WAL) & Rollback	Files are written to a temporary sibling file and committed via `os.replace()`. An atomic POSIX/NTFS syscall guarantees all-or-nothing writes.
Consistency	Schema constraints & foreign keys	Strict validation in Python layer: verified email formats, enum matching, and mandatory Rule 10 renewal date invariants.
Isolation	Transaction isolation levels	Granular file-level `threading.Lock()` prevents concurrent dirty reads or interleaved write streams.
Durability	Disk flush and battery cache	Synchronous `flush()` ensures data is written to underlying disk blocks before returning HTTP 200.

7. Core Backend Codebase & Module Specifications

7.1 File-by-File Functional Breakdown

Module File	Component Layer	Architectural Purpose & Key Methods
`app.py`	API Router & Server	Custom HTTP server subclassing `BaseHTTPRequestHandler`. Routes 35+ REST endpoints, parses URL queries, marshals JSON, and streams static assets.
`auth.py`	Security & Sessions	PBKDF2-HMAC-SHA256 hashing, session token creation (`secrets.token_hex`), token validation, password verification.
`config.py`	System Constants	Port binding (dynamic `$PORT`), file paths, quota defaults, and geopolitical zone definitions.
`state_zone_mapper.py`	Geopolitical Rules	Maps all 36 States/UTs to 6 statutory zones with whitespace and alias normalization.
`ai/decision_module.py`	Heuristics & Analytics	Heuristic evaluation generating 30/90/365-day milestone roadmaps, deficit formulas, and corridor detection.
`db/csv_engine.py`	Atomic Storage	Thread-safe flat-file engine with mutex locking, atomic file replacement, and schema validation.
`db/sync_engine.py`	Reconciliation Core	Two-way Node-to-Master synchronization, quota recalibration, and Rule 10 renewal date calculation.
`db/backup_engine.py`	Rolling Archiver	Autonomous 7-day ZIP backup generator, 2-version rolling retention pruner, and Google Drive webhook relay.
`db/issue_manager.py`	Issue Lifecycle	72-hour regional reminders, 7-day recurrent cycles, and 30-day central admin escalation engine.
`restore_backup.py`	CLI Disaster Recovery	Command-line utility for emergency system rollback from archive ZIP files.

7.2 Request Dispatching Flow in `app.py`

```
# Architectural flow within app.py
class NDLIRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urllib.parse.urlparse(self.path)
        path, query = parsed.path, urllib.parse.parse_qs(parsed.query)

        # 1. Static Asset Serving (CSS, JS, Images, PDFs)
        if path.startswith(("/static/", "/docs/images/")):
            return self.serve_static_file(path)

        # 2. Portal HTML Rendering
        if path in ("/", "/login", "/employee", "/admin", "/manual"):
            return self.serve_portal_template(path)

        # 3. REST API Dispatcher
        if path.startswith("/api/"):
            return self.dispatch_api_get(path, query)
```

8. Synchronization & Node-to-Master Reconciliation

8.1 Incremental Sync vs. Full Batch Reconciliation

The system utilizes a dual-mode synchronization model:

- **Real-Time Incremental Sync:** Triggered whenever a zonal officer submits an activity in SEC A or authorizes a club in SEC B/C. Writes directly to the local node CSV, then immediately invokes an atomic append to the master CSV file.
- **Full Node-to-Master Reconciliation:** An authoritative audit tool operated via `POST /api/sync/reconcile` (or the admin dashboard button). Designed for recovery after backup restoration or network partition.

8.2 Full Reconciliation Algorithm (`sync_engine.py`)

```
Step 1: Iterate over all provisioned employee directories (data/employees/emp01..07) Step 2: Read employee clubs.csv and activity_log.csv Step 3: Build in-memory lookup index of existing master_clubs.csv by Club ID Step 4: For each node club: - If Club ID not in master index: Append to master_clubs.csv (Zero Overwrite Loss) - Validate ISO-8601 timestamps & verify renewal date invariant (+1 year) Step 5: For each node activity: - If activity not in master_activities.csv: Append with atomic lock Step 6: Recalculate Quotas: - Count verified clubs per officer - Count verified support logs per officer - Overwrite master_quotas.csv with true reconstructed tallies Step 7: Return summary payload: { "reconciled_clubs": N, "reconciled_activities": M }
```

8.3 Quota Recalibration Guarantees

Because quotas are recalculated directly from raw historical transaction rows rather than incremented counters, any manual CSV edits or historical data imports are immediately harmonized without quota drift or double-counting.

9. Mathematical Models & Analytical Formulas

9.1 Employee Performance Star Rating Formula

The composite performance score of each Regional Zonal Officer is calculated dynamically via an approved multi-criteria weighting model:

$$\text{CompositeScore} = (\text{ClubsApproved} \times 0.50) + ((\text{TicketsClosed} + \text{PhoneCalls}) \times 0.30) + (\text{OfflineTraining} \times 0.20) + (\text{OnlineTraining} \times 0.10)$$

Weighting Rationale & Operational Justification

Activity Class	Weight	Strategic Objective
New Club Approvals	`0.50 (50%)`	Institutional expansion across secondary and higher education sectors.
Tickets Closed & Calls	`0.30 (30%)`	Operational SLA compliance and responsiveness to club grievances.
Offline Training	`0.20 (20%)`	Physical on-premise library orientation, workshops, and faculty training.
Online Training	`0.10 (10%)`	Virtual orientation webinars and student onboarding conclaves.

9.2 Benchmark Normalization & Star Assignment

The raw composite score is normalized against a monthly benchmark (`Benchmark = 50.0`) to derive the officer's performance star rating:

$$\text{NormalizedPercentage} = (\text{CompositeScore} / \text{Benchmark}) \times 100$$

Normalized Range	Star Rating	Appraisal Category	Administrative Action
80.0% – 100.0%+	⭐⭐⭐⭐⭐	**Outstanding**	Commendation & Best Practice Recognition
60.0% – 79.9%	⭐⭐⭐⭐	**Commendable**	Standard Operations Maintenance
40.0% – 59.9%	⭐⭐⭐	**Satisfactory**	Meets Minimum Expected Quotas
20.0% – 39.9%	⭐⭐	**Needs Improvement**	Targeted State Expansion Consultation
0.0% – 19.9%	⭐	**Critical Deficit**	Mandatory Central Administrative Review

10. Geopolitical State-Zone Mapping Specification

10.1 Territorial Jurisdiction Engine (`state_zone_mapper.py`)

To prevent administrative overlap or regional misclassification, the platform implements an automated **State-Zone Mapping Engine** covering all 36 States and Union Territories of India:

Statutory Zone	Assigned Node	Constituent States & Union Territories (Exhaustive List)
NORTH ZONE	EMP01	Jammu & Kashmir, Ladakh, Uttarakhand, Himachal Pradesh, Chandigarh, Punjab, Haryana, Delhi (NCT), Uttar Pradesh
CENTRAL ZONE	EMP02	Madhya Pradesh, Chhattisgarh
WEST ZONE	EMP03	Rajasthan, Gujarat, Maharashtra, Goa, Daman and Diu, Dadar & Nagar Haveli
EAST ZONE	EMP04	Bihar, Jharkhand, West Bengal, Odisha
NORTH EAST ZONE	EMP05	Sikkim, Assam, Arunachal Pradesh, Meghalaya, Manipur, Tripura, Nagaland, Mizoram
SOUTH ZONE 1	EMP06	Andhra Pradesh, Telangana, Karnataka (South Sector 1)
SOUTH ZONE 2	EMP07	Tamil Nadu, Puducherry, Kerala, Andaman & Nicobar Island, Lakshadweep (South Sector 2)

10.2 Normalization & Alias Matching Rules

The mapping engine incorporates string normalization to handle common typographic variations:

- Case-insensitive lookup with leading/trailing whitespace trimming.
- Automatic alias resolution (e.g., `"Delhi"` → `"Delhi (NCT)"`, `"J&K"` → `"Jammu & Kashmir"`).
- Deterministic exception raising on unmapped territorial strings, ensuring zero corrupted records in `master_clubs.csv`.

11. REST API Specification (Part 1: Auth & Operations)

11.1 Authentication Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
`POST`	`/api/auth/login`	Public	`{"identifier": "...", "password": "..."}` → `{"success": true, "token": "...", "user": {...}}`
`GET`	`/api/auth/me`	Bearer Token	Validates current session token. → `{"authenticated": true, "user": {...}}`
`POST`	`/api/auth/logout`	Bearer Token	Invalidates active session token from memory cache. → `{"success": true}`

11.2 Club Management Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
`GET`	`/api/clubs/search`	Employee/Admin	Query: `?q=delhi&zone=North` → `{"clubs": [...], "total": N}`
`POST`	`/api/clubs/create`	Employee/Admin	`{"id": "...", "reg_no": "...", "name": "...", "state": "...", ...}` → `{"success": true, "club": {...}}`
`POST`	`/api/clubs/update`	Employee/Admin	`{"id": "...", "updates": {...}}` → `{"success": true, "updated_fields": [...]}`
`POST`	`/api/clubs/renew`	Employee/Admin	`{"id": "...", "renewal_date": "YYYY-MM-DD"}` → `{"success": true, "next_renewal": "YYYY-MM-DD"}`

11.3 Support Activity Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
`POST`	`/api/activity/log`	Employee/Admin	`{"emp_id": "...", "support_type": "...", "notes": "..."}` → `{"success": true, "activity_id": "..."}`
`GET`	`/api/activity/list`	Employee/Admin	Query: `?emp_id=EMP01&limit=50` → `{"activities": [...], "total": N}`

12. REST API Specification (Part 2: Admin & System)

12.1 Central Administration Endpoints

Method	Endpoint Route	Auth Level	Functional Summary
`GET`	`/api/admin/metrics`	Admin	Aggregated KPIs, Donut Zone shares, Monthly Trajectory arrays, Support classifications.
`GET`	`/api/admin/employee-performance`	Admin	Evaluates all officers with 50/30/20/10 formula; filtered by year, month, and state.
`GET`	`/api/admin/renewal-attention`	Admin	Returns clubs with overdue or upcoming expiration (within 30 days).
`GET`	`/api/admin/ai-insights`	Admin	Generates automated strategic growth corridors and 30/90/365-day milestone roadmaps.
`POST`	`/api/admin/employees/create`	Master Admin	Provisions new employee node directory, dedicated CSVs, and PBKDF2 credentials.
`POST`	`/api/admin/employees/status`	Master Admin	Blocks or unblocks regional officer login access.
`GET`	`/api/admin/download/master-clubs`	Admin	Streams complete copy of `master_clubs.csv` with `Content-Disposition` attachment.

12.2 Backup, Sync & Disaster Recovery Endpoints

Method	Endpoint Route	Auth Level	Functional Summary
`GET`	`/api/admin/backup/status`	Admin	Returns metadata of last 2 retained snapshots, creation dates, and next backup due.
`POST`	`/api/admin/backup/trigger`	Admin	Instantly compiles manual ZIP snapshot, prunes old archives, and executes Drive relay.
`POST`	`/api/admin/backup/restore`	Dual-Layer	Validates Layer 1 CAPTCHA and Layer 2 Master Admin Password; decompresses and restores DBs.
`POST`	`/api/sync/reconcile`	Admin	Runs complete Node-to-Master batch audit across all 7 regional worker directories.

13. Backup Architecture & Disaster Recovery Pipeline

13.1 7-Day Autonomous Rolling Backup Daemon (`backup_engine.py`)

The platform features an autonomous background backup daemon running on a 7-day schedule. When triggered, it archives all critical database files into a timestamped, compressed ZIP snapshot:

```
# Archive manifest created by backup_engine.py
backup_20260915_180000.zip
├── master/
│   ├── master_clubs.csv
│   ├── master_activities.csv
│   ├── master_quotas.csv
│   ├── master_users.csv
│   └── master_issues.csv
└── employees/
    ├── emp01/ (clubs.csv, activity_log.csv, credentials.csv)
    ├── emp02/ ...
    └── emp07/ ...
```

13.2 Strict 2-Version Rolling Retention Invariant

To prevent unbounded disk consumption on cloud containers, the engine enforces a strict retention invariant: **only the 2 most recent snapshots are preserved**. All older snapshots are automatically unlinked:

```
def prune_old_backups(backup_dir: Path, max_retained: int = 2):
    backups = sorted(backup_dir.glob("backup_*.zip"), key=os.path.getmtime, reverse=True)
    for old_backup in backups[max_retained:]:
        try:
            old_backup.unlink()
            logger.info(f"Pruned historical snapshot: {old_backup.name}")
        except OSError as e:
            logger.error(f"Failed to prune snapshot: {e}")
```

13.3 Google Drive Webhook Relay Pipeline

Every generated backup is mirrored offsite to Google Drive via an HTTPS Webhook relay connecting to Google Apps Script. This provides redundancy against container failure or cloud host termination.

14. Emergency Restore Protocol & CLI Fallback

14.1 Web UI Emergency Restore Execution Flow

When executing an emergency restore through the Admin Portal interface:

1. **Trigger Dialog:** In Admin Portal → "Google Drive & Node Sync Health", click **"One-Click Restore Database ↺"**.
2. **Select Snapshot:** Choose the target backup archive from the dropdown (defaults to the latest snapshot).
3. **Solve Anti-Bot Challenge (Layer 1):** Enter the 5-character randomized alphanumeric code displayed on the Canvas.
4. **Re-Authenticate Password (Layer 2):** Enter the Master Administrator password (`admin@iitkgp.ac.in`).
5. **Atomic Decompression:** The server verifies credentials, decompresses the ZIP snapshot, replaces database tables via atomic mutex locking, and invokes automatic node reconciliation.

14.2 Server CLI Disaster Recovery Utility (`restore_backup.py`)

If the web server is offline or inaccessible, system administrators with SSH/terminal access can execute a direct command-line restore without web dependencies:

```
# Syntax: python restore_backup.py
$ python restore_backup.py data/backups/backup_20260915_180000.zip

======================================================================
NDLI Emergency Backup Restore Engine (CLI Utility)
======================================================================
[*] Verifying archive integrity: data/backups/backup_20260915_180000.zip ...
[+] Archive verified: 5 master files, 21 node files detected.
[*] Extracting database tables to scratch buffer ...
[*] Performing atomic replacement of master databases ...
[*] Performing atomic replacement of 7 regional employee node databases ...
[*] Triggering full Node-to-Master Reconciliation ...
[+] Reconciliation complete: 8 clubs, 14 activities synchronized.
======================================================================
[SUCCESS] Database restoration completed successfully. System online.
======================================================================
```

15. Front-End Architecture & Design System v3.0

15.1 Design Tokens & CSS Variables (`static/css/style.css`)

The v3.0 front-end architecture is built on a consolidated design token system engineered without bloated CSS frameworks:

```
:root {
  /* Brand Green Palette */
  --primary: #103125;
  --primary-light: #1A503E;
  --primary-accent: #0D7751;

  /* Complementary Amber CTA Accent System */
  --amber: #D97706;
  --amber-light: #F59E0B;
  --amber-bright: #FCD34D;
  --gradient-amber: linear-gradient(135deg, #B45309 0%, #D97706 55%, #F59E0B 100%);

  /* Glass Morphism Tokens */
  --surface-glass: rgba(255, 255, 255, 0.88);
  --surface-dark-glass: rgba(8, 22, 14, 0.84);
  --border-glass: rgba(255, 255, 255, 0.15);

  /* Radii & Elevation Hierarchy */
  --radius-sm: 6px;  --radius-md: 10px;  --radius-lg: 16px;
  --radius-xl: 20px; --radius-2xl: 28px; --radius-pill: 9999px;
  --shadow-sm: 0 2px 6px rgba(16,49,37,0.06);
  --shadow-lg: 0 16px 40px rgba(16,49,37,0.14);
  --transition-spring: all 0.35s cubic-bezier(0.34, 1.56, 0.64, 1);
}
```

15.2 Physics-Based Motion & Micro-Interactions

- **Spring Modals:** Styled with `animation: modalSpring 0.35s cubic-bezier(0.34, 1.56, 0.64, 1)` for a physical bounce pop.
- **Button Ripple:** Event-delegated click handler in `app.js` injecting a transient `<span class="ripple">` that animates outwards and cleans itself up automatically.
- **Scroll-Reveal Stagger:** Uses `IntersectionObserver` in `app.js` monitoring `.reveal-card` elements, adding `.reveal-visible` with 0.00s–0.22s cascaded delays.

16. Cloud Deployment & DevOps Infrastructure

16.1 Render Web Service Configuration (`render.yaml`)

The application is pre-configured for automated Git-driven deployments on Render.com or any container PaaS:

```
# render.yaml deployment configuration
services:
  - type: web
    name: ndli-club-management
    env: python
    buildCommand: "python -c "print('Build verification complete')""
    startCommand: "python app.py"
    envVars:
      - key: PYTHON_VERSION
        value: 3.10.12
      - key: STORAGE_MODE
        value: APPS_SCRIPT_RELAY
      - key: APPS_SCRIPT_RELAY_URL
        value: https://script.google.com/macros/s/.../exec
    disk:
      name: ndli-data-disk
      mountPath: /data
      sizeGB: 1
```

16.2 Dynamic Port Binding & Health Monitoring

- **Port Resolution:** The server dynamically reads `os.environ.get("PORT", 8080)`, ensuring seamless deployment behind reverse proxies (Nginx, Traefik, AWS ALB).
- **Health Check (`GET /api/health`):** Evaluates storage health, last backup timestamp, active nodes, and returns an HTTP 200 JSON payload for container liveness/readiness probes.

16.3 Google Apps Script (GAS) Serverless Deployment

The repository includes a complete mirror package in `deployment_gas/`: `Code.gs` (backend controller), `Landing.html`, `Admin.html`, and `Employee.html`. This allows 100% serverless deployment inside a Google Workspace domain without external cloud compute.

17. Automated Quality Assurance & Test Architecture

17.1 Test Suite Organization (132 Exhaustive Unit/Integration Tests)

The test suite located in `tests/` provides 100% functional and integration coverage across all system modules:

Test Suite Module	Tests	Verified Functional Areas
`test_api.py`	28 Tests	REST API routes, HTTP status codes, JSON payload schema validation, error handling.
`test_auth.py`	16 Tests	PBKDF2 encryption, 16-byte random salts, timing resistance, token expiry, session access.
`test_csv_engine.py`	18 Tests	Atomic writes, mutex locks, concurrent transactions, file corruption prevention.
`test_sync_engine.py`	22 Tests	Node-to-Master reconciliation, Rule 10 renewal dates (+1 year), quota updates.
`test_state_zone.py`	14 Tests	Exhaustive mapping of all 36 States/UTs to 6 statutory zones with alias normalization.
`test_features_1_to_5.py`	24 Tests	Master CSV download, 7-day rolling backups, 72h reminders, 30d escalations, restore security.
`test_portal_and_api.py`	10 Tests	Headless Node.js template script evaluation, DOM event execution, view transitions.

17.2 Executing the Full Test Suite

```
# Command to run full test suite with standard output verification
$ python -m unittest discover tests -v

----------------------------------------------------------------------
Ran 132 tests in 44.447s

OK (132 tests passing, 0 failures, 0 errors)
```

18. System Maintenance, Operations & Troubleshooting Runbook

18.1 Routine Maintenance Operations

- **Periodic Master Reconciliation:** Run `POST /api/sync/reconcile` monthly or after manual CSV edits to ensure zero discrepancies across worker nodes.
- **Manual Backup Generation:** Click **"Trigger Manual Backup Now"** before applying operating system updates or migrating server environments.
- **Audit Log Review:** Inspect server console logs for repeated failed login attempts or unauthorized REST API calls.

18.2 Operational Troubleshooting Runbook

Observed Symptom	Root Cause Diagnostic	Remediation Procedure
`401 Unauthorized` on login	Invalid PBKDF2 password or inactive employee account.	Verify `is_active` flag in `master_users.csv`; re-provision credentials via Admin Portal.
Quota count mismatch in UI	Node CSV entries logged during network disruption.	Click **"Run Full Node-to-Master Reconciliation"** in Admin Portal to recalculate quotas.
Backup storage growing	Pruning daemon disabled or permissions error.	Verify file write permissions on `data/backups/`; verify 2-version retention pruner.
Lock gate flashing on load	Missing synchronous pre-unlock DOM class.	Ensure synchronous `auth-pre-unlocked` script block in `<head>` is preserved intact.

19. Security Audit, Data Integrity & Compliance

19.1 OWASP Top 10 Security Posture Assessment

OWASP Risk Category	System Vulnerability Defense	Compliance Implementation Status
A01: Broken Access Control	Server-side RBAC validation on all REST routes; session tokens verified per request.	`COMPLIANT` No client-side bypass possible.
A02: Cryptographic Failures	PBKDF2-HMAC-SHA256 with 100,000 rounds; 16-byte random salts; TLS mandatory in cloud.	`COMPLIANT` Industry standard encryption.
A03: Injection Attacks	No SQL database engine used (zero SQLi attack surface); CSV fields sanitized & escaped.	`COMPLIANT` Injection vector eliminated.
A05: Security Misconfig	Zero external pip dependencies; minimal attack surface; no debug flags in production.	`COMPLIANT` Hardened native runtime.
A07: Identification Failures	Dual-layer security challenge for destructive actions; rate-limited session verification.	`COMPLIANT` Two-factor visual verification.

19.2 Data Transparency Declaration

The architecture guarantees that all institutional club records, regional officer interaction logs, and administrative decisions remain stored in human-readable, portable, vendor-neutral CSV formats. Data can be imported into any spreadsheet or analytical tool without specialized database conversion tools.

20. System Handover Sign-Off & Architectural Attribution

20.1 Technical Deliverables Handover Certification

This document certifies the formal technical handover of the **NDLI Club Management and Employee Activity Tracking System (Release v3.0)**. All source code, database architectures, REST API contracts, test automation suites, and cryptographic models documented herein are fully implemented, verified, and operational.

Deliverable Component	Verification Standard	Handover Status
Full Source Codebase	Zero external Python dependencies; 35+ REST endpoints	`VERIFIED & HANDED OVER`
Cryptographic Security Core	PBKDF2-HMAC-SHA256 with 100k rounds; 16-byte random salts	`VERIFIED & HANDED OVER`
Automated Test Suite	132/132 unit and integration tests passing cleanly	`VERIFIED & HANDED OVER`
Disaster Recovery Engine	Dual-layer security restore; 7-day rolling backups	`VERIFIED & HANDED OVER`
Modern Minimalist UI (v3.0)	Manrope font, amber accents, glass morphism, spring motion	`VERIFIED & HANDED OVER`
20-Page Operational User Manual	Complete step-by-step guides with original live screenshots	`VERIFIED & HANDED OVER`

20.2 Official Attribution & Engineering Endorsement

Lead System Architect & Full-Stack Developer

Dr. Anirban Mukherjee

Midnapore
West Bengal 721101, India

v4.2 Migration Toolkit (API & data notes)

The migration toolkit for legacy clubs (v4.2) adds the following surfaces. All validation is server-side; the UI merely previews it.

- `record_mode` **on club creation:** "import" accepts explicit historical dates (`date_of_approval` required, `last_renewal_date` optional, window 2019-01-01 … today with formation ≤ last renewal ≤ today); "new" auto-stamps and REJECTS explicit dates.
- `record_origin` **schema field (CLUB_FIELDS):** "imported" or "created" — record provenance alongside the entry timestamp.
- `POST /api/clubs/bulk-import/validate` **:** dry-run — per-row report (READY/ERROR), writes nothing; duplicates (reg-no/club-id) blocked against the master DB and within the batch; 200 rows max.
- `POST /api/clubs/bulk-import/commit` **:** creates only valid rows via `approve_new_club(confirm_cloud_sync=False)` followed by ONE batched durable-sync confirmation.
- `POST /api/clubs/correct-dates` **(admin-only, logged):** the only path that alters historical dates — validates the same window, recomputes the upcoming renewal (+1 year anchor), and appends a "Data Correction" activity row. The general club-update path treats historical dates as immutable.

v5.0 Data Counting, Migration & Operations (API & data notes)

Release v5.0 makes counting honest and explicit, and adds the operating surfaces for the two bulk doors, the employee history migration, the PPT training library and the Hard Reset handover. All validation is server-side; the UI merely previews it. This section is the data-transparency reference for how records are counted, attributed and handled.

- **Counting rule and fields:** every club record carries `record_origin` — `"created"` (live work, counted) or `"imported"` (migrated history, visible but never counted as new work). Dashboard totals split accordingly: `GET /api/admin/metrics` returns `summary.total_clubs`, `summary.live_clubs` and `summary.migrated_totals` (baseline grand totals from the migrated worksheets, scoped by year/zone); the employee performance payload returns `live_clubs_approved` / `migrated_clubs_approved` alongside the `live_*` / `migrated_*` activity splits, and the profile payload carries `live_clubs_approved_count`. The human rule: *"N = X live clubs + Y migrated club approvals (history)"* — the migrated half is the history BASELINE grand totals, never the imported register rows (one bucketing rule everywhere: register records are the per-club detail and are never added on top of the baseline, so nothing is double counted).
- **Door 1 — employee bulk NEW clubs:** **`POST /api/clubs/bulk-import/validate`** | **`/commit`:** `emp_id` + `rows` (JSON array, 200 rows max). This door creates NEW work: `record_mode="new"` → `record_origin="created"`, counted live. Dates are auto-stamped; explicit past dates are refused by the value-aware date guard (a new club cannot be back-dated), duplicates (reg-no / club-id) are blocked against the master DB and within the batch. `/commit` creates only validated rows via `approve_new_club(confirm_cloud_sync=False)` followed by ONE batched durable-sync confirmation. `/validate` is a free dry-run that writes nothing.
- **Door 2 — admin bulk club HISTORY:** **`POST /api/admin/migration/clubs/validate`** | **`/commit`:** admin-only, `rows` ≤ 200, each row requires `approved_by_emp_id` (the officer who approved the club) so migrated history keeps its attribution. This door is HISTORY: `record_mode="import"` → `record_origin="imported"`, zero live credit, historical dates accepted inside the 2019-01-01 … today window. Dual-layer security: `/commit` requires the Master Admin password re-authentication (a session alone never authorises migration); `/validate` stays password-free. These are **the two doors**: admin Data Migration (history, not counted) versus employee Bulk Add New Clubs (new work, counted).

- **Employee activity history migration:** `POST /api/admin/migration/history/validate` | `/commit` : admin-only; `emp_id` + `csv_text` (raw worksheet contents). The dry-run reports grand totals, per-employee attribution, year ranges and worksheet gap checks. `/commit` accepts `batch_id`, `replace` (true = rebaseline) and `actor`, persists the baseline and immediately recomputes the master quota so baseline + live counts show at once. Master Admin password is required on commit only.
- **PPT Training Library:** two slots (15 MB total). `GET /api/training/files` (slot metadata), `GET /api/training/download/<slot>` (employee download), `POST /api/admin/training/upload` (admin-only: `slot` "1"|"2", `filename`, base64 `data`; `.ppt` / `.pptx` / `.ppsx` only, magic-byte checked) and `POST /api/admin/training/delete`. Upload replaces the slot's previous deck. The slots are wiped by Hard Reset — re-upload after handover.
- **Hard Reset (handover):** `POST /api/admin/data/hard-reset` : admin-only and deliberately hard to press — three locks: `confirm="HARD-RESET-FOR-HANDOVER"`, the Master Admin password, and the 6-digit handover PIN (checked as a SHA-256 digest *before* any checkpoint work, so a wrong PIN can never reach the wipe). An automatic checkpoint makes the operation abort-safe. It wipes clubs, activities, quotas, issues, migration history, Robu chat/learning logs and the training slots — everything except the provisioned accounts — and reseeds nothing.
- **Date corrections:** `POST /api/clubs/correct-dates` remains the only path that alters historical dates (see the v4.2 notes above). The general club-update path treats historical dates as immutable.

National Digital Library of India (NDLI) Club Management System
Enterprise Technical Reference Manual & System Handover Architecture Specification (v5.0)
Architected & Engineered by **Dr. Anirban Mukherjee** • Indian Institute of Technology Kharagpur
© 2026 NDLI IIT Kharagpur. All Rights Reserved. Confidential & Proprietary Handover Document.

Appendix A. Auto-Generated API Reference & Architecture Map

This appendix is **generated from the code** by `gen_api_docs.py` straight from the dispatch tables in `routes/` -- it can never disagree with the router. Hand-written request/response semantics stay in sections 11-12; module prose stays in section 7. Regenerate after any route change (`python3 gen_api_docs.py`), and run the drift gate (`--check`) in CI / the test suite.

A.1 Component Map (measured from the tree)

Path	Role	Size
`app.py`	HTTP request handler + startup wiring (no dispatch chain)	509 lines
`routes/`	URL router: ordered dispatch tables + 16 domain modules	17 files, 3595 lines, 81 dispatch entries
`db/`	flat-file CSV storage engine (mutex, backup, migration, counting)	9 modules, 3227 lines
`ai/`	Robu assistant (KB retrieval, tools, LLM gateway, guard, audit)	11 modules, 4731 lines
`auth.py`	PBKDF2 password hashing + session tokens	430 lines
`config.py`	environment-driven configuration + seed accounts	200 lines
`state_zone_mapper.py`	state/zone normalization + alias matching	147 lines
`qr_tokens.py`	certificate QR token signing / verification	151 lines
`wsgi.py`	WSGI entry point for cloud hosting	129 lines
`init_db.py`	storage hierarchy + master schema bootstrap	325 lines
`restore_backup.py`	CLI disaster-recovery restore utility	81 lines
`templates/`	portal HTML pages (admin, employee, index, verify)	4 files
`static/`	front-end assets (css / js / images)	68 files, 44 JS
`docs/`	technical + user manuals (Robu KB sources) + figures	4 manual files + figures
`tests/`	the pinned gate (unittest, zero third-party deps)	82 test modules
`data/`	runtime flat-file store (CSV, sessions, backups, logs) -- runtime state, deliberately not counted	runtime state

A.2 Route Module Map (generated from routes/)

Module	Responsibility	Dispatch entries	Lines
`routes/activity.py`	Activity log: list, log, delete	3	135
`routes/assistant.py`	Robu - the AI assistant surface	10	272
`routes/auth_routes.py`	Authentication & session endpoints	5	139
`routes/backup.py`	Backup status / trigger / restore	3	94
`routes/certificate.py`	Certificates: settings, signatures, QR tokens, public verification	6	319
`routes/clubs.py`	Club register: search, details, create, update, renew, delete	7	407
`routes/data_admin.py`	Destructive data-admin (hard reset)	1	91
`routes/downloads.py`	File downloads (manuals, promo video, CSV exports)	5	171
`routes/employees.py`	Employee roster / profile / admin employee management	6	260
`routes/health.py`	Health + state/zone mapping	3	44
`routes/issues.py`	Issue tracking	6	160
`routes/metrics.py`	Admin metrics, performance, renewal attention, AI insights	4	587
`routes/migration.py`	Bulk import + data migration (the two doors)	3	398
`routes/portals.py`	HTML portals + static assets	9	98
`routes/sync_routes.py`	Sync status / reconcile / Drive sync	6	104
`routes/training.py`	PPT Training Library (2 slots)	4	146

Total: 81 dispatch entries (46 GET + 35 POST) covering 116 distinct paths.

A.3 Request Dispatch Flow

```
HTTP request
  |
  v
app.py  NDLIRequestHandler.handle_get / handle_post        [app.py]
  |        parse URL + query / JSON body
  v
routes.match(GET_ROUTES | POST_ROUTES, path)               [routes/__init__.py]
  |        ordered table, FIRST match wins (E = exact, P = prefix)
  v
routes/<module>.<handler>(h, path, query_params | body)    [routes/<domain>.py]
  |        domain logic: db/ (CSV store), ai/ (Robu), auth.py, ...
  v
h._send_json(...)  /  h._send_file(...)  /  h._send_error(...)   -> response
```

A.4 GET Endpoints (dispatch order) (1/2)

Path legend: no suffix = exact match, `*` = path-prefix match. Rows are in dispatch order -- the first matching row wins.

#	Path	Match	Handler	Module
1	`/favicon.ico`	exact	`portals.get_favicon`	`portals`
2	`/static/*` `/docs/*`	prefix	`portals.get_static_assets`	`portals`
3	`/portal` `/index.html`	exact	`portals.get_index_portal`	`portals`
4	`/`	exact	`portals.get_root`	`portals`

5	`/manual`
	`/user-manual`
	`/documentation`
	`/user_manual`
	`/manual/view`
		exact	`portals.get_user_manual_page`	`portals`

6	`/admin`	exact	`portals.get_admin_portal`	`portals`
7	`/login` `/employee/login`	exact	`portals.get_login_portal`	`portals`
8	`/employee` `/employee/*`	exact+prefix	`portals.get_employee_portal`	`portals`

9	`/verify`
	`/certificate/verify`
	`/verify-certificate`
		exact	`portals.get_verify_portal`	`portals`

10	`/api/health`	exact	`health.get_health`	`health`

11	`/api/state-zone/map`
	`/api/states`
	`/api/states-zones`
		exact	`health.get_state_zone_map`	`health`

12	`/api/state-zone/lookup`	exact	`health.get_state_zone_lookup`	`health`
13	`/api/auth/me`	exact	`auth_routes.get_auth_me`	`auth_routes`
14	`/api/employees/roster`	exact	`employees.get_employees_roster`	`employees`
15	`/api/employees/profile`	exact	`employees.get_employees_profile`	`employees`
16	`/api/clubs/search`	exact	`clubs.get_clubs_search`	`clubs`
17	`/api/activity/list`	exact	`activity.get_activity_list`	`activity`
18	`/api/admin/metrics`	exact	`metrics.get_admin_metrics`	`metrics`
19	`/api/admin/employee-performance`	exact	`metrics.get_admin_employee_performance`	`metrics`
20	`/api/admin/renewal-attention`	exact	`metrics.get_admin_renewal_attention`	`metrics`
21	`/api/clubs/details`	exact	`clubs.get_clubs_details`	`clubs`
22	`/api/admin/employees`	exact	`employees.get_admin_employees`	`employees`
23	`/api/admin/ai-insights`	exact	`metrics.get_admin_ai_insights`	`metrics`
24	`/api/assistant/nudges`	exact	`assistant.get_assistant_nudges`	`assistant`
25	`/api/assistant/insights`	exact	`assistant.get_assistant_insights`	`assistant`
26	`/api/assistant/interactions`	exact	`assistant.get_assistant_interactions`	`assistant`

A.4 GET Endpoints (dispatch order) (2/2)

#	Path	Match	Handler	Module
27	`/api/assistant/audit`	exact	`assistant.get_assistant_audit`	`assistant`
28	`/api/assistant/privacy`	exact	`assistant.get_assistant_privacy`	`assistant`
29	`/api/sync/status`	exact	`sync_routes.get_sync_status`	`sync_routes`

30	`/api/download/user-manual`
	`/api/download/manual`
	`/download/user-manual`
	`/download/manual`
	`/manual.pdf`
	`/NDLI_Club_Management_User_Manual.pdf`
	`/user-manual.pdf`
		exact	`downloads.get_download_user_manual`	`downloads`

31	`/api/download/promo-video`
	`/api/download/video`
	`/download/promo-video`
	`/download/video`
	`/ndli_promo_video.mp4`
	`/promo-video.mp4`
	`/video.mp4`
	`/promo-video`
	`/video`
		exact	`downloads.get_download_promo_video`	`downloads`

32	`/api/admin/download/master-clubs` `/api/admin/download/master_clubs.csv`	exact	`downloads.get_download_master_clubs`	`downloads`
33	`/api/admin/sync-issues`	exact	`sync_routes.get_admin_sync_issues`	`sync_routes`
34	`/api/admin/backup/status`	exact	`backup.get_backup_status`	`backup`
35	`/api/employee/download/activity-log` `/api/employee/download/activity_log.csv`	exact	`downloads.get_download_activity_log`	`downloads`
36	`/api/employee/download/clubs-log` `/api/employee/download/clubs.csv`	exact	`downloads.get_download_clubs_log`	`downloads`
37	`/api/issues/employee-reminders`	exact	`issues.get_issues_employee_reminders`	`issues`
38	`/api/issues/admin-reminders`	exact	`issues.get_issues_admin_reminders`	`issues`
39	`/api/issues/list`	exact	`issues.get_issues_list`	`issues`
40	`/api/issues/details`	exact	`issues.get_issues_details`	`issues`
41	`/api/certificate/settings`	exact	`certificate.get_certificate_settings`	`certificate`
42	`/api/certificate/signature`	exact	`certificate.get_certificate_signature`	`certificate`
43	`/api/certificate/verify`	exact	`certificate.get_certificate_verify`	`certificate`
44	`/api/admin/drive/sync-now`	exact	`sync_routes.get_drive_sync_now`	`sync_routes`
45	`/api/training/files`	exact	`training.get_training_files`	`training`
46	`/api/training/download/*`	prefix	`training.get_training_download`	`training`

A.5 POST Endpoints (dispatch order) (1/2)

Path legend: no suffix = exact match, `*` = path-prefix match. Rows are in dispatch order -- the first matching row wins.

#	Path	Match	Handler	Module
1	`/api/assistant/ask`	exact	`assistant.post_assistant_ask`	`assistant`
2	`/api/assistant/feedback`	exact	`assistant.post_assistant_feedback`	`assistant`
3	`/api/assistant/outcome`	exact	`assistant.post_assistant_outcome`	`assistant`
4	`/api/assistant/preferences`	exact	`assistant.post_assistant_preferences`	`assistant`
5	`/api/assistant/forget`	exact	`assistant.post_assistant_forget`	`assistant`
6	`/api/auth/login`	exact	`auth_routes.post_auth_login`	`auth_routes`
7	`/api/auth/logout`	exact	`auth_routes.post_auth_logout`	`auth_routes`
8	`/api/auth/change-password`	exact	`auth_routes.post_auth_change_password`	`auth_routes`
9	`/api/auth/verify-password`	exact	`auth_routes.post_auth_verify_password`	`auth_routes`
10	`/api/activity/log`	exact	`activity.post_activity_log`	`activity`
11	`/api/employee/activity/delete`	exact	`activity.post_activity_delete`	`activity`
12	`/api/clubs/bulk-import/validate` `/api/clubs/bulk-import/commit`	exact	`migration.post_bulk_import`	`migration`
13	`/api/admin/migration/clubs/validate` `/api/admin/migration/clubs/commit`	exact	`migration.post_migration_clubs`	`migration`
14	`/api/admin/migration/history/validate` `/api/admin/migration/history/commit`	exact	`migration.post_migration_history`	`migration`
15	`/api/clubs/create`	exact	`clubs.post_clubs_create`	`clubs`
16	`/api/admin/data/hard-reset`	exact	`data_admin.post_hard_reset`	`data_admin`
17	`/api/admin/clubs/delete`	exact	`clubs.post_admin_clubs_delete`	`clubs`
18	`/api/clubs/update`	exact	`clubs.post_clubs_update`	`clubs`
19	`/api/clubs/correct-dates`	exact	`clubs.post_clubs_correct_dates`	`clubs`
20	`/api/clubs/renew`	exact	`clubs.post_clubs_renew`	`clubs`
21	`/api/admin/employees/status`	exact	`employees.post_admin_employees_status`	`employees`
22	`/api/admin/employees/create`	exact	`employees.post_admin_employees_create`	`employees`
23	`/api/admin/employees/update`	exact	`employees.post_admin_employees_update`	`employees`
24	`/api/sync/reconcile`	exact	`sync_routes.post_sync_reconcile`	`sync_routes`
25	`/api/admin/backup/trigger` `/api/admin/backup/create`	exact	`backup.post_backup_trigger`	`backup`
26	`/api/admin/backup/restore` `/api/admin/backup/emergency-restore`	exact	`backup.post_backup_restore`	`backup`

A.5 POST Endpoints (dispatch order) (2/2)

#	Path	Match	Handler	Module
27	`/api/issues/create`	exact	`issues.post_issues_create`	`issues`
28	`/api/issues/resolve`	exact	`issues.post_issues_resolve`	`issues`
29	`/api/certificate/qr-token`	exact	`certificate.post_certificate_qr_token`	`certificate`
30	`/api/certificate/signature`	exact	`certificate.post_certificate_signature`	`certificate`
31	`/api/certificate/settings`	exact	`certificate.post_certificate_settings`	`certificate`
32	`/api/admin/drive/sync-now`	exact	`sync_routes.post_drive_sync_now`	`sync_routes`
33	`/api/admin/drive/pull-now` `/api/sync/pull-now`	exact	`sync_routes.post_drive_pull_now`	`sync_routes`
34	`/api/admin/training/upload`	exact	`training.post_training_upload`	`training`
35	`/api/admin/training/delete`	exact	`training.post_training_delete`	`training`